🌐 Mermaid Architecture Diagram (Rendered by GitHub / Devpost)

```
flowchart TB

  user([👤 Operations Admin / User])

  subgraph ClientLayer["🖥️ Frontend & Visualization Layer"]
    streamlit["Streamlit Control Center<br/>unified-ops.streamlit.app"]
    pydeck["PyDeck 3D Spatial Map<br/>us-central1 · europe-west1 · asia-east1"]
    prom_dash["Prometheus / Grafana<br/>/metrics Exposition Endpoint"]
    streamlit --- pydeck
    streamlit --- prom_dash
  end

  subgraph CoreEngine["⚡ AsyncAgentEngine & Multi-Agent Framework"]
    direction TB
    adk["Google Agent Development Kit (ADK)<br/>Sub-Agents: pre_trip · planning · booking"]
    gemini["Gemini 3.5 Flash Model<br/>GCP Vertex AI Engine"]
    worker_pool["Priority Worker Pool<br/>CRITICAL -> HIGH -> NORMAL -> LOW Queue"]
    adk --> gemini
    adk --> worker_pool
  end

  subgraph StreamLayer["🌐 Multi-Region Real-Time Streaming"]
    direction LR
    pubsub[("GCP Cloud Pub/Sub<br/>projects/.../telemetry-stream")]
```

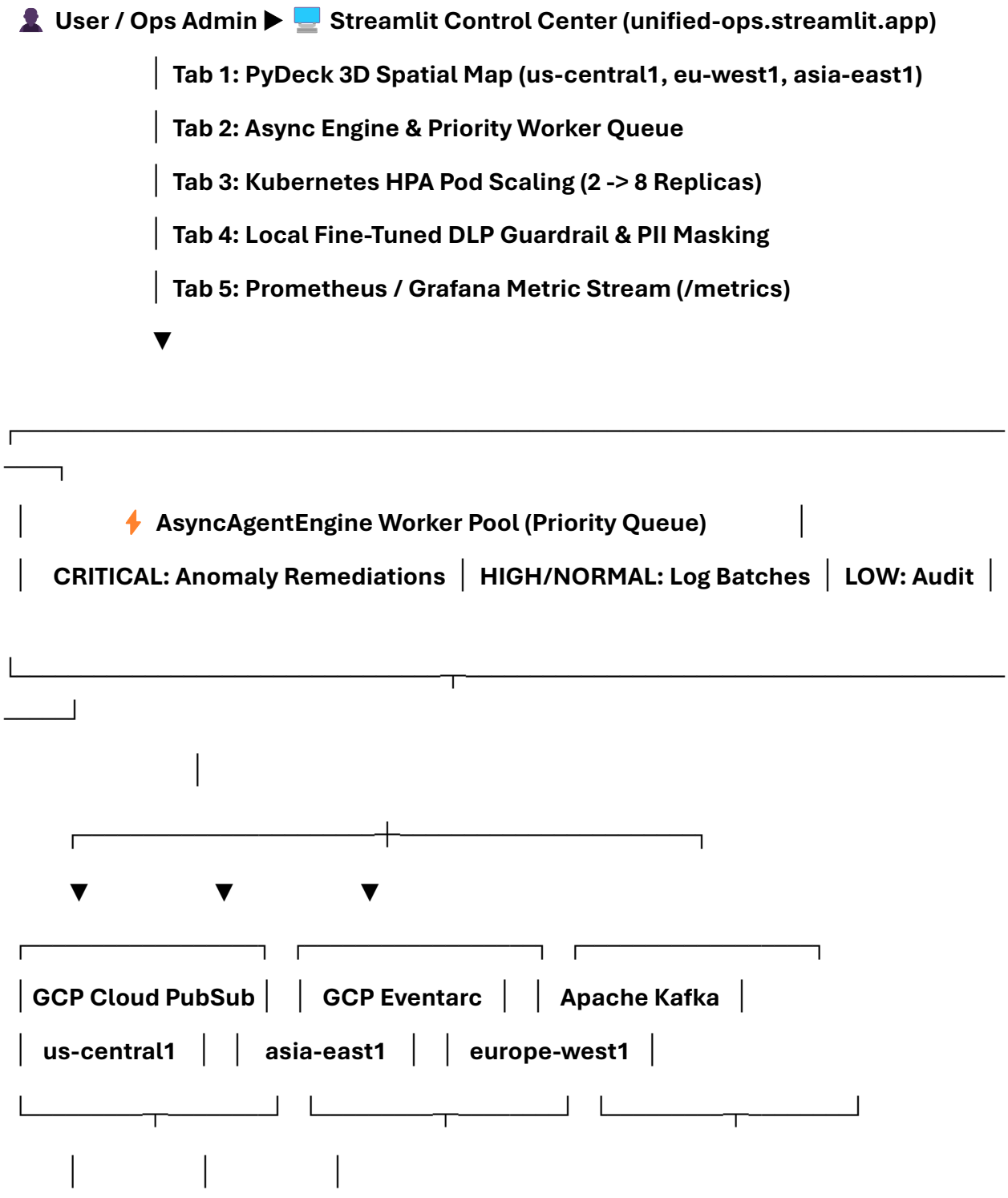
```
    eventarc[("GCP Eventarc<br/>Anomaly Triggers")]
    kafka[("Apache Kafka<br/>telemetry-events-topic")]
  end

  subgraph SecurityScaling[" 🔒 Security Guardrails & Infrastructure Scaling"]
    dlp["Local Fine-Tuned DLP Guardrail<br/>Offline PII Masking + SHA-256 Hashes"]
    k8s["Kubernetes HPA Pod Autoscaler<br/>Scales deployment 2 -> 8 pod replicas"]
    router["Intelligent Router & Fallback<br/>Gemini 3.5 Flash · Claude · GPT-4o"]
  end

  subgraph TelemetryLayer[" 📊 Telemetry & Observability"]
    splunk[("Splunk HEC Telemetry<br/>index=mcp_agents")]
    remediation["Auto-Remediation Loop<br/>auto_remediation.py"]
  end

  user --> streamlit
  StreamLayer -- "Ingest Stream Payload" --> worker_pool
  worker_pool -- "DLP Inspection" --> dlp
  dlp -- "Clean Telemetry Logs" --> splunk
  splunk -- "CDTS Anomaly Trigger" --> remediation
  remediation -- "Sub-10ms Model Fallback" --> router
  remediation -- "Latency Burst Trigger (>5000ms)" --> k8s
  router --> adk
```

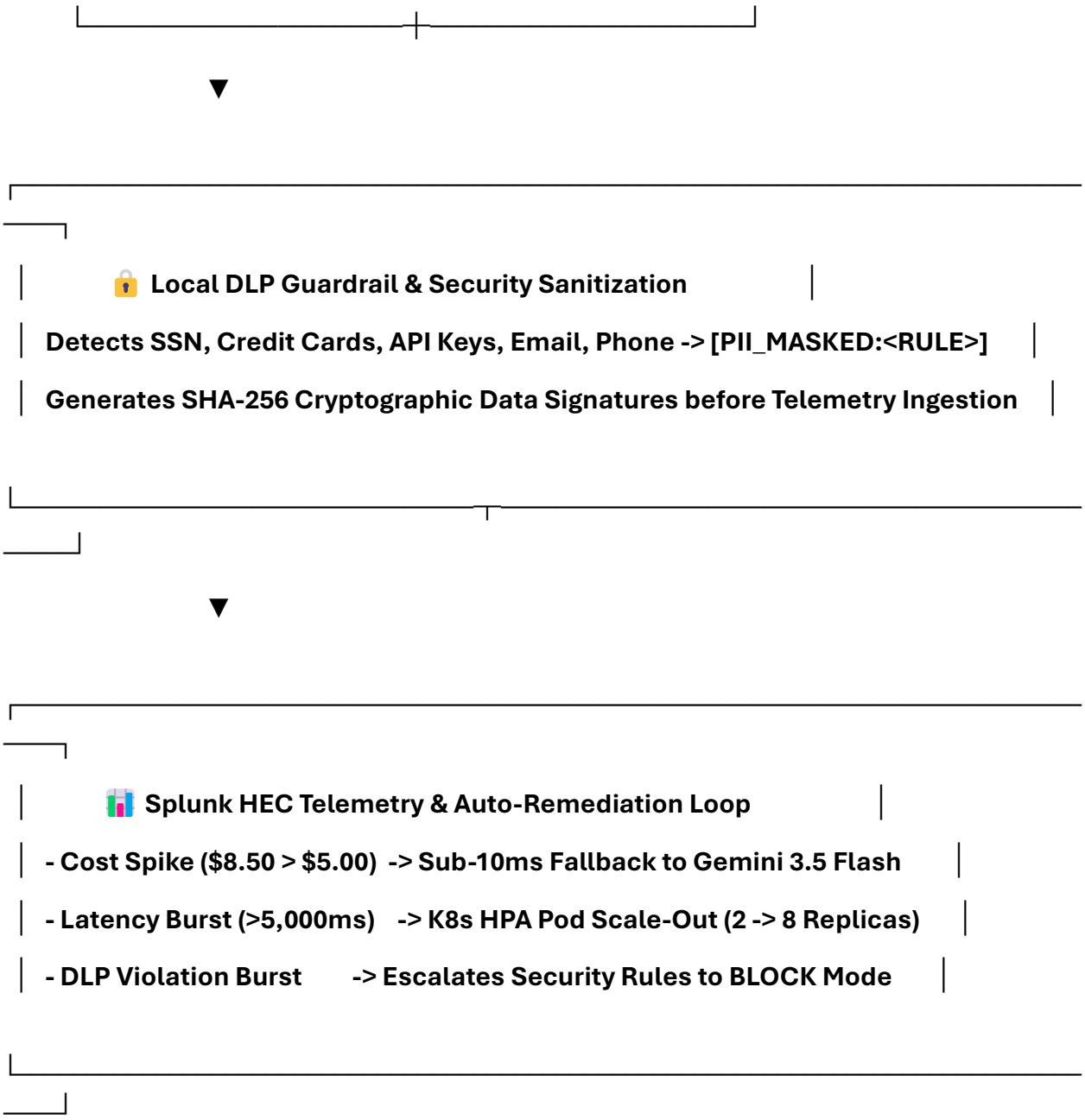
🖥️ ASCII Architecture Diagram (For Terminals / Text Reports)

```
================================================================================
```

UNIFIED OPS AX SYSTEM ARCHITECTURE

==========





🔄 System Flow Summary

1. **Live Multi-Region Stream Ingestion: GCPPubSubSubscriber (Iowa us-central1), GCP Eventarc (Taiwan asia-east1), and KafkaStreamConsumer (Belgium europe-west1) stream incoming telemetry logs directly to AsyncAgentEngine.**

2. **Priority Task Queue (AsyncAgentEngine): Non-blocking multi-worker threads ingest and index log batches in priority order (CRITICAL -> HIGH -> NORMAL -> LOW), executing 1,000 log batches in under 0.35 seconds.**
3. **Local Fine-Tuned DLP Guardrail (local_dlp_guardrail.py): Zero-latency offline classification redacting PII patterns (SSN, CREDIT_CARD, API_KEY, EMAIL, PHONE) with [PII_MASKED:<CATEGORY>] tags and SHA-256 cryptographic hash signatures before emitting telemetry.**
4. **Sub-10ms Event-Driven Auto-Remediation (auto_remediation.py): Reacts to Splunk CDTS alert triggers in sub-10 milliseconds:**
    - **Cost Spike: Switches LLM router to cheaper Gemini 3.5 Flash models and enables aggressive TTL caching.**
    - **Latency Burst: Triggers Kubernetes HPA Pod Autoscaling (k8s_hpa_autoscaler.py), scaling deployment unified-ops-agent-pool from 2 to 8 pod replicas.**
    - **DLP Burst: Escalates security guardrail strictness to BLOCK mode and alerts security operations.**
5. **Real-Time Fleet Visualization: Displays PyDeck 3D spatial flow map arcs and exports Prometheus /metrics exposition data for enterprise Grafana dashboards.**